

课程笔记

[CS 153] 数据基础设施与国防级扩展 — Palantir CTO Shyam Sankar

基于 Stanford CS 153 课程内容整理

2025



视频作者/频道: Stanford CS 153

发布日期: 2025-04-28

视频时长: 44:03

视频链接: <https://www.youtube.com/watch?v=jB13kCmWT2k>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：从第 13 号员工到 CTO	3
1.1 硅谷对国防科技的态度演变	3
1.2 本章小结	3
2 基础设施演进：从 On-Prem 到异构云	4
2.1 早期的 On-Prem 挑战	4
2.2 异构性：Palantir 独特的扩展挑战	4
2.3 本章小结	4
3 Apollo：软件交付基础设施	5
3.1 Pre-Apollo 的黑暗时代	5
3.2 Apollo 的设计哲学	5
3.3 Log4j 案例：自动化的威力	6
3.4 本章小结	6
4 Rubik's：多租户 Kubernetes 编排器	6
4.1 工程师认知负担问题	6
4.2 容器安全的不可变策略	6
4.3 解放应用层的实验速度	7
4.4 本章小结	7
5 Project Maven 与算法战争	7
5.1 OODA 循环与算法战争的起源	7
5.2 从传感器到目标：计算机视觉的应用	7
5.3 决策链的逐层深入	8
5.4 威慑的揭示与隐藏	8
5.5 本章小结	8
6 政府与商业：决策链的统一抽象	8
6.1 Kill Chain、Value Chain 与 Decision Chain	8
6.2 企业软件的幂律分布	9
6.3 本章小结	9
7 制造业软件栈的困境与 Warp Speed	9
7.1 软件工业综合体的批判	9
7.2 BMW 案例：乌克兰战争的供应链冲击	10
7.3 Warp Speed 平台与新型国防制造	10
7.4 造船业的战略意义	10
7.5 本章小结	11
8 AI 的供需动态	11
8.1 本章小结	11

9 二战启示与现代制造竞争	11
9.1 本章小结	12
10 隐私、伦理与效率前沿	12
10.1 Privacy and Civil Liberties Engineering	12
10.2 本章小结	12
11 IPO 后的变化与制度性反思	12
11.1 上市的冲击	12
11.2 方向盘比喻：制度失灵的根源	13
11.3 本章小结	13
12 总结与延伸	13
12.1 拓展阅读	14

1 引言：从第 13 号员工到 CTO

本次 CS 153 讲座邀请了 Palantir Technologies 的 CTO 兼执行副总裁 Shyam Sankar。Sankar 于公司成立初期以第 13 号员工的身份加入，在 Palantir 工作了 19 年，亲历了公司从 Stanford 边缘项目成长为全球最大的数据基础设施企业之一的全过程。

Palantir Technologies 背景

Palantir 由 Peter Thiel、Alex Karp 等人于 2003 年联合创立，总部位于 Denver，核心业务是为政府（国防/情报）和商业企业提供大规模数据集成与分析平台。公司名取自《指环王》中的 Palantír（真知晶球），寓意“洞察全局”。截至讲座时，Palantir 拥有约 1000 个生产环境、5000+ 微服务，政府与商业业务各占约 50%。

Sankar 本科毕业于 Cornell（2003 届），彼时正值 Post-Dotcom 时代，大多数 CS 同学选择了咨询或银行业，唯一来校招的科技公司是 Microsoft，连 Google 都不来。Sankar 来到 Stanford 读研作为进入硅谷的跳板，在两个月内成为汇款公司 Xoom 的第五号员工，由此结识了 PayPal Mafia 圈层（Kevin Hartz、Roelof Botha 等）。

国家安全的驱动力

Sankar 出生于印度，幼年随家庭在尼日利亚生活。一次严重的武装抢劫事件迫使全家以难民身份移居美国 Orlando。父亲虽未再商业成功，却因家人安全而满足。这段经历使 Sankar 对国家安全怀有深厚的个人使命感。911 事件后他曾尝试退学加入 CIA，最终在 Palantir 找到了将技术与国家安全结合的理想归宿。Sankar 指出：“加入 Palantir 之后我就‘废了’（ruined）——还能去做什么？开发一个 Ajax 日历应用吗？”

1.1 硅谷对国防科技的态度演变

Sankar 回顾了硅谷对 Palantir 所做工作的抵触经历两个阶段：

1. **政治冷漠期（2003–2013）**：知名 VC 们认为为政府做软件是“浪费人才”，建议 Palantir 转做银行软件。这并非政治立场，而是对政府问题的漠不关心。
2. **政治对立期（2013–2020s）**：硅谷的主流叙事变为“政府是邪恶的，不应该为其服务”，上升为明确的政治声明。

硅谷的国防历史

Sankar 提醒听众一个常被遗忘的事实：1950 年代，Lockheed Martin 曾是硅谷最大的雇主。硅谷的起源本身就与国防科技密不可分。如今硅谷对国防工作态度的转变，在某种意义上是一种历史回归。

1.2 本章小结

Sankar 的个人经历——从移民家庭到 PayPal Mafia 到国防科技——为理解 Palantir 的企业文化和技术选择提供了关键背景。硅谷对国防科技从冷漠到对立再到回归的态度转变，是 Palantir 成长历程的重要外部环境。

2 基础设施演进：从 On-Prem 到异构云

2.1 早期的 On-Prem 挑战

Sankar 指出，Palantir 创立时（2005–2006）AWS 甚至尚未发布 S3，根本不存在所谓的“互联网基础设施栈”。一切必须部署在本地（on-prem），而且客户环境是**气隙隔离**（air-gapped）的——不连接互联网，不能从 Palo Alto 办公室 SSH 连入。

Air-Gapped 环境的严苛约束

在气隙环境中：（1）无法远程访问，必须物理到场；（2）申请新硬件需要约 6 个月的采购周期；（3）一切都是分类信息（classified），合规要求极高。这意味着传统的“随时扩容”思维完全不适用，资源优化的需求从第一天就极为强烈。

当时所有公司都在构建**大型单体应用**（monolithic applications），Scale-out 架构还是新概念，主要限于 Google 等少数公司的内部研究。Palantir 的所有客户都在隔离环境中运行，SRE 必须**前置部署**（forward deploy）到客户现场，人力消耗巨大且无法规模化。

2.2 异构性：Palantir 独特的扩展挑战

Sankar 强调，Palantir 的扩展挑战不仅仅是“规模大”：

异构性才是核心难题

Palantir 不是拥有少数几个多租户云栈，而是拥有 **1000 个生产环境**，其中 **100 个是气隙隔离**的。这些环境的形态极其多样：有的运行在 M1 Abrams 坦克上，有的在潜艇中，有的是大型 EU 多租户云栈。工程师必须编写能在这种异构性上运行的代码。Sankar 指出：“不是我们的基础设施有多大的问题——任何时刻我们的云基础设施可能在创建和销毁 20,000 个 Pod——而是它有多**异构**。”

这种异构性体现在多个维度：

维度	范围	挑战
网络连通性	完全联网 ↔ 气隙隔离	无法假设网络可达
硬件形态	坦克/潜艇 ↔ 云数据中心	资源差异数量级
合规等级	商业云 ↔ 分类网络	控制措施不同
连接持续性	始终在线 ↔ 月级离线	潜艇下水数月
租户模式	多租户共享 ↔ 单客户专用	隔离与效率权衡

表 1: Palantir 生产环境的异构维度

2.3 本章小结

Palantir 面临的不是典型的“单一巨型集群如何扩展”问题，而是“如何在 1000 个形态各异的环境中保持软件的一致性和可运维性”。这一独特挑战催生了 Apollo 和 Rubik's 两大内部平台。

3 Apollo: 软件交付基础设施

3.1 Pre-Apollo 的黑暗时代

Sankar 将 2006–2014 年描述为“Pre-Apollo 黑暗时代”。在这个阶段：

- 软件以**季度大版本**（quarterly releases）形式发布，2006–2009 年间每季度只发布一次
- 升级时，工程师需要带着多张 DVD（包含所有二进制文件）物理前往客户环境
- SSH 进入后，需要在脑中管理微服务间的依赖关系，按正确顺序升级
- 出错时需要回滚，可能导致整个升级窗口失败

升级地狱（Upgrade Hell）与任务关键约束

在伊拉克和阿富汗等早期部署中，所有军事行动在夜间进行，工程师只有**白天的窗口**来执行升级。如果在窗口内无法完成，**必须回滚**——因为软件宕机意味着任务受阻。Sankar 描述道：“你感觉整个世界的重量都压在你的肩上。”环境漂移（drift）不可避免：一个版本落后就意味着 6 个月的差距，工程师不得不维护古老的代码。

3.2 Apollo 的设计哲学

Apollo 始建于 2014 年，花了约 7 年时间逐步完善。其核心思想是用**机器人（robots）**替代人工 SRE：

Apollo 的声明式方法

Apollo 在产品（微服务）和环境之间建立了一层**抽象**。工程师以声明式方式描述：

- 我的依赖项和所需的其他服务版本
- 升级策略（如 Blue-Green Deployment）
- 健康度量指标和 Soak Time（如 5 分钟浸泡时间）
- 自动回滚条件

打包完成后交给 Apollo，它会自动编排 Canary Path、执行 Blue-Green 升级、提供全套可观测性和遥测数据。工程师只需专注于写代码和发布，无需关心 1000 个环境的细节。

Apollo 的关键能力数据：

- 管理约 **5,000 个离散微服务**
- 跨越 **1,000 个生产环境**（100 个气隙隔离）
- 每周执行约 **100,000 次升级**
- 自动处理约 **20,000 次/周**的 CVE 驱动回滚

3.3 Log4j 案例：自动化的威力

Sankar 用 Log4j 漏洞事件来说明 Apollo 的价值：

Log4j 漏洞响应对比

大多数企业在 Log4j 漏洞爆发时，首先面临的问题是不知道哪些环境运行了受影响的库版本——连清查漏洞面都做不到。而 Apollo 精确记录了每个环境中每个软件的每个版本。此外，Apollo 内置了 CVE 感知能力：当新 CVE 发布且存在已知安全版本时，**自动回滚到安全版本**，无需工程师介入。对于 Log4j，Palantir 在发布补丁后 **24 小时内**完成了全部 1000 个环境的部署。

危机中的代码冻结悖论

传统思维认为危机时应进入“代码冻结”（Code Freeze）——变更带来风险。但 Sankar 指出这是一种反模式。以阿富汗撤离为例，当时根本不存在乘客登记软件，如果进入代码冻结，就只能用 Excel 管理撤离名单。Palantir 的做法相反：**危机时代码速率反而增加**，因为“软件是你最灵活的武器系统”。正常每周 100,000 次升级，危机时会更多。

3.4 本章小结

Apollo 是 Sankar 在 Palantir 最引以为豪的基础设施成就。它将手动的、人力密集的升级流程转化为声明式、自动化的交付流水线，使 Palantir 在最严苛的异构环境中实现了高频持续交付。Apollo 的核心洞察是：在产品与环境之间建立抽象层，让工程师只需描述“我要什么”，而非“怎么做”。

4 Rubik's：多租户 Kubernetes 编排器

4.1 工程师认知负担问题

Sankar 指出，如果工程师需要了解 Kubernetes 在 on-prem 和云端的所有差异，“速度就归零了——知识负担太重”。Rubik's 作为 Palantir 的**多租户 Kubernetes 编排器**，在所有异构环境之上提供了统一的抽象。

Rubik's 的核心设计原则

Rubik's 是产品工程团队与底层基础设施之间的核心抽象层。它的设计目标是：让工程师完全不需要关心底层是 on-prem Kubernetes 还是云端 Kubernetes、是受限的军事环境还是商业云——所有差异由 Rubik's 屏蔽。Palantir 离职校友最怀念的两样东西就是 Apollo 和 Rubik's。

4.2 容器安全的不可变策略

在高度受监管的合规环境中，Rubik's 采用了一种激进的安全策略：

72 小时容器生命周期

Rubik's 将所有容器镜像视为**不可变** (immutable)。每个容器的最长存活时间为 **72 小时**，在 40–72 小时之间被**随机销毁**。重新启动时，从完全打过补丁的新镜像拉取。Sankar 解释道：“不做补丁 (patching)，而是做替换。对于 APT (Advanced Persistent Threat) 或国家级攻击者来说，你必须每 72 小时重新攻破每台主机——祝你好运。”

将合规控制编码为软件

传统做法是用人工流程 (convention) 来执行合规控制——3 个环境还能勉强应付，1000 个环境就必须**将所有控制编码进软件**。Rubik's 实现了声明式的 Ingress/Egress 管理、安全策略自动执行等。这意味着从**第一次 commit** 开始，代码就已经满足生产就绪 (production-ready) 的合规要求。Sankar 将其总结为：“把尽可能多的生产就绪性 (production readiness) 从**惯例**移入软件。”

4.3 解放应用层的实验速度

Rubik's 层是最不受客户定制影响的基础层——它是所有软件运行其上的抽象。有了这个稳定的地基，应用层可以获得极高的实验自由度：工程师不需要担心“我的原型能否安全部署到这个环境”，因为安全、合规、网络隔离等问题已经在 Rubik's 层被系统性地解决了。

4.4 本章小结

Rubik's 解决了异构 Kubernetes 环境下工程师的认知负担问题，通过不可变容器、自动合规和环境抽象，将生产就绪性从人工惯例转化为软件保障，从而在高安全环境中释放了应用层的开发速度。

5 Project Maven 与算法战争

5.1 OODA 循环与算法战争的起源

Sankar 介绍了 Project Maven (2017 年启动) 的背景，引用了 John Boyd 的 **OODA 循环** (Observe-Orient-Decide-Act) 理论。

OODA 循环的历史

John Boyd 最初为理解美军在空战中被苏联 MiG 战斗机压制的原因而提出 OODA 循环。核心洞察：**谁能更快地完成“观察-定向-决策-行动”循环，谁就赢**。问题不在于美军飞行员技术差，而在于机体不够灵活，无法快速迭代循环。Sankar 指出，这一框架同样适用于软件基础设施：加速决策链就是加速 OODA 循环。

2017 年时，美国国防部将此称为“算法战争” (Algorithmic Warfare)，核心问题是：如何利用 AI 实现更强的威慑力？

5.2 从传感器到目标：计算机视觉的应用

Sankar 用一个生动的类比解释了挑战的规模：

全球入境 vs 天基传感器

使用 Global Entry 时，你的面部占传感器视野的 80%。而天基传感器拍摄地球照片时，你要找的目标不到视野的万分之二。传统方式是让分析员手动在 10 英里 × 10 英里的照片中平移搜索——如同“在灯光下找钥匙”。2017 年计算机视觉成熟后，终于可以自动搜索导弹发射井、运输起竖发射车（TEL）等目标。

5.3 决策链的逐层深入

Sankar 描述了 Maven 从 2017 年至今的演进，每解决一个层次的问题就暴露出下一层：

1. **发现问题**：如何在 24 小时内找到 1000 个目标？（计算机视觉）
2. **处理问题**：找到目标后，用什么武器？派哪个单位？法律审查（JAG）如何签字？如何确认正面识别（Positive ID）？
3. **后勤约束**：发现弹药不足，如何做预测性补给（Predictive Resupply）？
4. **反事实遗憾**：如果今天打了这个目标，明天会不会有更有价值的目标出现？如何评估反事实遗憾（Counterfactual Regret）？

动态打击的时间尺度

美军军事教条中，计划时间低于 **96 小时** 的打击即被视为“动态打击”（Dynamic Strike）。历史上军事规划一直假设有充足的“审慎规划”（Deliberate Planning）时间。现实是面对对手的突发动作，响应能力严重不足。Sankar 指出：如果你在 OODA 循环上比对手快 10 倍，这本身就是强大的威慑。

5.4 威慑的揭示与隐藏

Sankar 讨论了威慑理论中的核心悖论：**对手必须知道你的能力，威慑才有效**。军方通过定期演习来选择性地展示和隐藏能力——展示是为了威慑，隐藏是为了保留战略突然性。主持人将此类比为科技行业的 WWDC Keynote：向开发者生态展示即将到来的能力。Sankar 认为这是个不错的比喻，“只不过是真实世界中执行的”。

5.5 本章小结

Project Maven 展示了 AI/CV 技术在国防领域的具体应用路径。关键洞察是：技术问题是逐层暴露的——解决了感知层，决策层成为瓶颈；解决了决策层，后勤层成为约束。端到端优化整条决策链（Kill Chain），而非局部优化某个环节，才是真正的竞争优势。

6 政府与商业：决策链的统一抽象

6.1 Kill Chain、Value Chain 与 Decision Chain

Sankar 揭示了 Palantir 政府与商业业务之间的深层联系：

三条链的统一抽象

- 政府称之为 Kill Chain：从传感器到射手（Sensor to Shooter）
- 商业称之为 Value Chain：从供应商到客户（Hand of Supplier to Hand of Customer）
- 计算机科学家看到的是 **Decision Chain**：一系列决策，每个决策约束下游选项

Palantir 的核心能力是优化这条决策链的整体效率——加速 OODA 循环。政府与商业业务看似截然不同，但底层抽象是同构的。

商业业务占 Palantir 收入的 50%，客户包括 BMW（全系车型）、Chrysler、HD Hyundai（造船）、Airbus（飞机制造）等。Sankar 指出合规要求在两端并无本质差异——美国保险业在 50 个州分别受监管，可观测性和合规数据的要求与政府客户相当。

跨行业知识迁移：BP 到 Warp Speed 的故事

Sankar 讲述了一个知识迁移的典型案列：Palantir 两周内交付了 Operation Warp Speed（COVID 疫苗分发供应链），但这之所以可能，是因为两年前为 BP 解决了**结构上完全相同的问题**——优化烃类生产。Sankar 引用 Steve Jobs 的话：“你只能在回顾时连接这些点。”但他当时就知道为 BP 解决的问题极有价值。这正是 Palantir 50/50 政府-商业模式的战略意义。

6.2 企业软件的幂律分布

Sankar 区分了消费者软件与企业软件的产品策略差异：

高斯分布 vs 幂律分布

消费者软件的需求接近**高斯分布**——从大量用户中取平均，找到信号最强的需求来构建。企业软件的需求则是**幂律分布**：某个客户今天独有的问题，可能是所有客户 5 年后都会面临的问题。关键在于用**产品品味**识别哪些“边缘”问题蕴含普遍价值，然后通过**归纳法**（induction）从 1 次解、3 次解中总结出通用方案。

6.3 本章小结

Palantir 的竞争壁垒在于将 Kill Chain、Value Chain 统一抽象为 Decision Chain，并通过跨行业部署积累知识迁移能力。50/50 的政府-商业模式不是业务分散，而是一个正向飞轮：每个行业的经验都是其他行业的研发投入。

7 制造业软件栈的困境与 Warp Speed

7.1 软件工业综合体的批判

Sankar 对现有企业软件行业提出了尖锐批评：

可卖的软件 vs 有价值的软件

Sankar 指出存在一个“软件工业综合体” (Software Industrial Complex): 公司制造的软件**旨在好卖** (sellable), 而非真正**有价值** (valuable)。COVID 是一次残酷的实战检验——企业投入了数百亿美元建设供应链软件栈, 结果两周内全面崩溃。CEO 们的财报电话中唯一提到拯救了他们的 IT 投资是什么? **Zoom 和 Teams**——视频会议软件。而那些花了巨资的供应链系统如同“纸老虎”。

Sankar 描述了亲眼所见的现实: 走到 Airbus 的工厂车间, 发现实际的飞机制造流程居然在 **Excel** 中完成。工人们逃离了那些“花哨的软件”, 因为那些软件解决的不是他们的实际问题。

7.2 BMW 案例: 乌克兰战争的供应链冲击

5 美元线束引发的工厂危机

乌克兰战争爆发后, BMW 一种从乌克兰采购的 5 美元线束出现了间歇性供应中断。BMW 拥有“精美的 SAP 实现”, 但系统被设计为从左到右运行: 客户订单 → 物料清单。突然间需要**反向运行**: 基于库存来决定优先生产哪些订单 (7 系利润率远高于 1 系)。SAP 做不到这一点, 慕尼黑工厂面临停产风险。Palantir 在**几天内**用软件解决了这个问题。

7.3 Warp Speed 平台与新型国防制造

Palantir 将 15 年来在制造业积累的经验打包为 **Warp Speed** 平台, 目标是赋能新一代制造企业。Sankar 特别提到:

- Anduril 每周有 1/8 的员工在使用 Palantir
- Shield AI、Epirus、Saronic 等新兴国防科技公司正在基于 Warp Speed 构建自动化工厂
- HD Hyundai (全球第二大造船企业) 的全流程——从船舶设计到维修——都由 Palantir 驱动

CI/CD for Manufacturing

Sankar 将 Warp Speed 的核心理念类比为**制造业的 CI/CD**。传统上“生产”仅指制造已设计好的产品, 但现代做法是将设计与制造紧密耦合——SpaceX/Starlink 的工程师坐在工厂车间, 观察组装过程并实时修改设计以提高可制造性 (Design for Manufacturability)。Warp Speed 将 PLM、供应链、生产线数据**端到端集成**, 使设计决策能立即看到对零件报废、供应链、生产周期的影响。

7.4 造船业的战略意义

250:1 的造船产能差距

中国的造船产能是美国的约 **250 倍**。美国正在退役比建造更多的舰船——如同“生育率危机”。问题不仅是产能不足, 更在于**仍在用旧方式建造**。Sankar 认为解决方案不在底特律 (传统工业基地), 而在 El Segundo (现代国防科技聚集地), 需要用 SpaceX/Tesla 式的垂直整合思维来重构造船业。

7.5 本章小结

现有制造业软件栈的核心问题是**断裂的决策链**——设计、供应链、生产之间缺乏实时反馈。Warp Speed 通过端到端集成重建了这条链，其商业制造经验直接反哺国防制造现代化。

8 AI 的供需动态

Sankar 对 AI 发展提出了一个重要的供需框架：

AI 的价值在需求侧，不在供给侧

当前市场过度关注 AI 供给侧（模型训练）。但如果把 AI 类比为电力，历史上价值并未主要积累在发电公司，而是在**利用电力创造经济价值的机器公司**。Sankar 认为美国作为国家的最大机会在于加速 AI 的**需求侧**——用 AI 解决真正重要的问题（医疗、国防、制造等）。”这些问题在 ChatGPT 出现之前就存在，现在你拥有了新形式的电力去解决它们。”

Sankar 呼吁应该有”10 个曼哈顿计划”来用 AI 解决医疗等核心问题，而不是把所有注意力都放在模型本身。

中国竞争的双重叙事

Sankar 引用 Peter Thiel 的观察：中国的宣传历来有两种看似矛盾的形式——”别费劲了，我们已经赢了”和”别费劲了，我们不是威胁”。两者的共同目的是让对手**停止竞争**。Sankar 强调，现实是”我们每天都在竞争”，关键不仅是限制对手获取技术，更重要的是**每天比对手更快地部署和使用这些技术**。

8.1 本章小结

AI 的战略价值在于应用（需求侧），而非模型（供给侧）。国家竞争力的关键是谁能更快、更深入地将 AI 部署到实际决策链中。

9 二战启示与现代制造竞争

Sankar 从历史视角审视了当前的国防制造竞争态势。

二战的工厂战争

二战前夕，美国刚刚掌握大规模生产能力。Bill Knudsen——丹麦移民、前 Ford 和 GM 高管——被直接授予三星将军军衔，负责战争动员。二战本质上是一场**工厂之战**：德国人是更好的工程师，制造精美但数量少的产品；美国制造的产品不那么精美，但**数量碾压**。Sankar 的警告：如今角色已**反转**——我们的对手是大规模生产的高手，而美国在制造”精美但稀少”的东西。

中国国防承包商的多元化模式

中国国防承包商仅 27% 的收入来自解放军 (PLA)，其余 73% 来自商业市场——消费者在 Amazon 上购买的廉价商品可能正在**交叉补贴**军事杀伤力。这正是美国曾经获胜时的模式：Ford 造汽车也造卫星（直到 1990 年），General Mills 的麦片公司造鱼雷（利用其谷物加工机械部门的能力）。Sankar 认为重建这种多元化的研发基础至关重要。

9.1 本章小结

历史证明，制造能力是战争的决定性因素。当前美国面临制造产能劣势，但通过软件驱动的现代制造（El Segundo 模式 vs Detroit 模式），可以实现不对称优势。

10 隐私、伦理与效率前沿

10.1 Privacy and Civil Liberties Engineering

Sankar 介绍了 Palantir 创立之初就建立了**隐私与公民自由工程团队**——这是最早的工程团队之一。

Palantir 的隐私哲学：效率前沿

Alex Karp（极左）和 Peter Thiel（极右）因在 Stanford 法学院宿舍辩论而成为朋友，共同认为：恐怖主义本身很糟糕，但**社会对恐怖主义的反应可能更糟**——通常导致公民自由的反射性剥夺和监控国家的形成。Sankar 用**效率前沿**（Efficient Frontier）来理解这一问题：隐私与安全之间存在权衡，政治辩论的是应该在前沿上的哪个位置，而**工程的使命是将整条前沿向外推**——使得在任何给定的隐私水平上，都能获得更多安全。

将辩证法化为工程问题

Sankar 承认纯粹的自由意志主义和纯粹的社会主义都不可行——极端化是学术意义上的。真正的进步在于：不是在极端之间摆钟，而是通过**技术能力**使两端的权衡变得不那么尖锐。主持人评论这是他第一次听到用“Pareto 最优前沿”来讨论伦理问题。Sankar 回应：“这很实际（practical）。”

10.2 本章小结

Palantir 的伦理框架不是回避权衡，而是承认权衡的存在并用工程手段**扩展可能性空间**。这与整个讲座的主题一致：通过更好的软件基础设施，使得看似不可调和的矛盾变得可以同时优化。

11 IPO 后的变化与制度性反思

11.1 上市的冲击

Sankar 坦言 Palantir 2020 年上市带来了“错位体验”（dislocating experience）：

- **周期压缩**：私有时以年为单位思考，上市后被迫以季度为单位——每 13 周损失 1 周来服务投资者

- **沟通失衡**：上市后一度过于偏重外部沟通，导致内部员工失去”知情特权感”。私有时期所有沟通面向内部，上市后钟摆摆向另一极端
- **文化韧性**：尽管有这些冲击，Sankar 认为公司文化本身并未改变，改变的是运营节奏和沟通方式

11.2 方向盘比喻：制度失灵的根源

迪士尼丛林巡航的方向盘

Sankar 对当代制度失灵的诊断：无论是企业 C-Suite 还是政府机构的领导者，都在认真地转动方向盘。问题是，这个方向盘是**迪士尼丛林巡航游乐项目中的道具**——没有连接到任何东西。基层员工抬头说”这些人怎么这么蠢？”，但根本原因是**缺乏 OODA 循环**：决策层没有实时反馈，执行层的真实情况无法上达。当人们逃离企业软件去 Excel 中解决问题时，整条反馈链就断裂了。

Sankar 警告两种错误应对方式：一种是虚无主义（”制度都不行，推倒重来”），这行不通；另一种是维持现状。正确的态度是”每天起来让这些制度比昨天更好一点”——这正是 Palantir 软件试图实现的。

11.3 本章小结

上市改变的是运营节奏，而非公司使命。制度失灵的根本原因是决策链断裂——解决方案不是推倒制度，而是用软件重建从”方向盘”到”车轮”的连接。

12 总结与延伸

Shyam Sankar 的讲座围绕一个核心主题展开：**如何在极端异构、高安全性、任务关键的环境中构建和运维软件基础设施**。关键收获包括：

1. **抽象是扩展的关键**：Apollo（产品-环境抽象）和 Rubik's（Kubernetes 异构抽象）使 5000 个微服务在 1000 个异构环境中实现了每周 100,000 次自动升级。
2. **声明式 > 命令式**：工程师描述”我要什么”，系统自动解决”怎么做”。这同时降低了认知负担和操作风险。
3. **危机中加速而非冻结**：软件是最灵活的武器系统。Apollo 的自动化使得危机时代码速率可以提升而非下降。
4. **Decision Chain 是统一抽象**：Kill Chain、Value Chain 本质上都是 Decision Chain。跨行业经验可以迁移，50/50 模式是战略优势而非分散。
5. **将合规编码为软件**：1000 个环境无法用人工惯例保证合规，必须将控制逻辑自动化。72 小时容器生命周期是安全与运维的优雅融合。
6. **AI 的价值在需求侧**：模型训练是供给侧，真正的竞争力在于谁能更快地将 AI 部署到实际决策中。
7. **效率前沿作为伦理框架**：用工程能力扩展隐私-安全的可能性空间，而非在固定前沿上做零和博弈。

12.1 拓展阅读

- Palantir Apollo 技术博客: <https://blog.palantir.com/>——了解 Apollo 和 Rubik's 的公开技术细节
- John Boyd, *Patterns of Conflict*——OODA 循环的原始理论框架
- *Arsenal of Democracy* (A.J. Baime)——关于二战美国工业动员的历史著作, 了解 Bill Knudsen 的故事
- Project Maven 相关报道——搜索 “Project Maven algorithmic warfare” 了解 AI 在军事中的应用争议与进展
- Stanford CS 153: Infrastructure at Scale 课程主页——其他讲座涵盖 Cloudflare、Uber 等公司的基础设施实践